# Dependency Free Error Detection

Zach Myers and Mark Horowitz

Stanford University

2023
February

**Abstract**

We describe how our checker is able to detect errors by only checking the errors against limited number of error patterns. Our description uses a simpler, table-based algorithm to derive these results. We show that our detection scheme achieves MLSD equivalent performance for a subset of single precursor channels with three check patterns. We show how this result is related to the cross-correlation between our check patterns and the errors we are checking. We show how we achieve near-MLSD checker performance for a much larger subset of single precursor channels. We then extend this result to two precursor channels. For the single precursor case, we will describe an analytical algorithm to determine what check patterns are necessary to achieve MLSD equivalent performance for a particular channel. For both the single and double precursor channels, we will describe a computational algorithm to determine what check patterns are necessary for near MLSD equivalent performance for a large comprehensive set of channels. We show how the use of an FFE to produce a bit estimate correlates errors and change which check patterns are required for BER optimal performance.

# Contents

**1 Feed Forward Error Detection** **3**

1.1 The Residual Estimated Error (REE) . . . . . 5

1.2 REE Error Detection . . . . . 6

**2 Check Patterns** **11**

2.1 Complete Detection . . . . . 12

2.2 Partial Detection . . . . . 14

2.3 Single Precursor Channel Coverage . . . . . 16

2.4 Effective Coverage . . . . . 17

2.5 Two Precursor Channel Coverage . . . . . 20

**3 Check Patterns for Practical Bit Estimators** **24**

**Appendices** **28**

**A Glossary Of Terms** **29**

**B DFE** **30**

**C +,0+ Error Pattern** **32**

**D Smaller $h_{-2}$** **34**

**E Modified System** **36**

# Chapter 1

# Feed Forward Error Detection

Building the signal processing back-ends to ADC based high-speed links requires a careful balancing act. While close to optimal signal detection (maximal likelihood sequence detection, or MLSD) and equalization (either DFE, or Viterbi for MSLD) are well known, these techniques depend on knowing the prior bit decision before making the next decision. This "backward" dependency is difficult[1] to implement at very high bit rates, so other, sub-optimal, techniques must be used. These usually involve periodically breaking the backward dependency which allows parallelization of filter. Once this parallelism exists, multiple copies of the filter can be built, and each filter can run slower than the incoming bit rate. Of course breaking this dependency increases the resulting hardware overhead and error rate.

This paper describes a different approach to address this problem. Rather than trying to build a fast, accurate detector, we build a possibly noisy detector, and a very accurate checker. Our key insight is that checking is easily parallelized, since when checking a bit string all bits are already known — even the bits that will be transmitted after the current bit. Thus checkers can optimally correct for both pre and post cursor inter-symbol-interference (ISI). We use this insight to create a system which yields MSLD quality signal detection at modest DSP hardware costs. A block diagram of our proposed approach is shown in Figure E.1.

[1]Techniques to extend the time for this dependency typically grow exponentially with increasing time

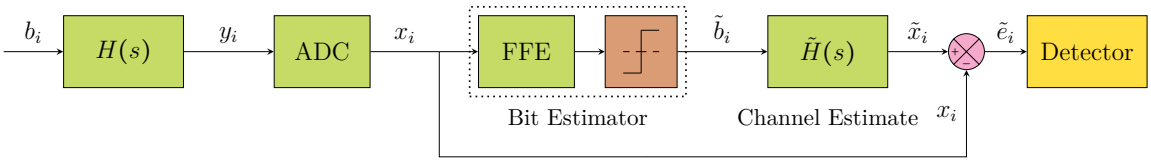



Figure 1.1: Block Diagram Of Error Detection Scheme

Our "noisy" detector takes the output of the ADC and feeds it into a feedfoward equalizer (FFE) and a slicer to create a pretty good estimate of the transmitted bits, $b_i$. The bit estimate, $\tilde{b}_i$, is then convolved with a channel estimate, $\tilde{H}(s)$, to make an estimate of our sampled input, $\tilde{x}_i$. If we subtract this estimated input sequence, $\tilde{x}_i$, from our actual ADC samples, $x_i$, we create a error signal rich in information, $\tilde{e}_i$. This signal contains information about bit errors, phase miss-estimation, channel miss-estimation and the sampled noise. We call this signal the residual estimated error (REE). While the REE signal contains a rich source of information, this paper will focus solely on using it to correct bit errors. We will assume standard LMS (least mean squared) loops are used to adapt the FFE and channel estimation parameters to their optimal values.

We simplify the design of our error detector by reducing its required function: it only needs to detect possible errors. It doesn't need to determine what the correct bit sequence should be. Assuming that the error rate of our estimator is lower that $10^{-2}$, a single Viterbi decoder can handle all sequences which contain a possible error to generate the correct output. Our detector then operates off the REE searching for errors. To ensure it has the same quality as the Viterbi engine, it also uses sequence detection.

As shown in Figure E.1, our detection scheme is clearly feedforward up until the detector block. We will show in Section 1.2, that, for error checking, we can implement the needed sequence detection that is also feedforward. And we will show in Section 2.1 that this checker achieves near parity with a MLSD-based detection scheme. Before addressing those topics, the next section describes the effect erroneous bit estimates have on the REE.

## 1.1 The Residual Estimated Error (REE)

The REE is the difference between the received input, $x_i$, and our best estimate of what that input should be, $\tilde{x}_i$ ($\tilde{b}_i$ convolved with $\tilde{H}(s)$) given our estimates of the received bits, $\tilde{b}_i$. Notice that this signal does not contain any noise amplification from the FFE. In fact, the signal is independent from the FFE except for the misestimated bits it causes. Assuming a good channel estimate, when the bit estimates are correct the REE is just the total input referred noise in the system.

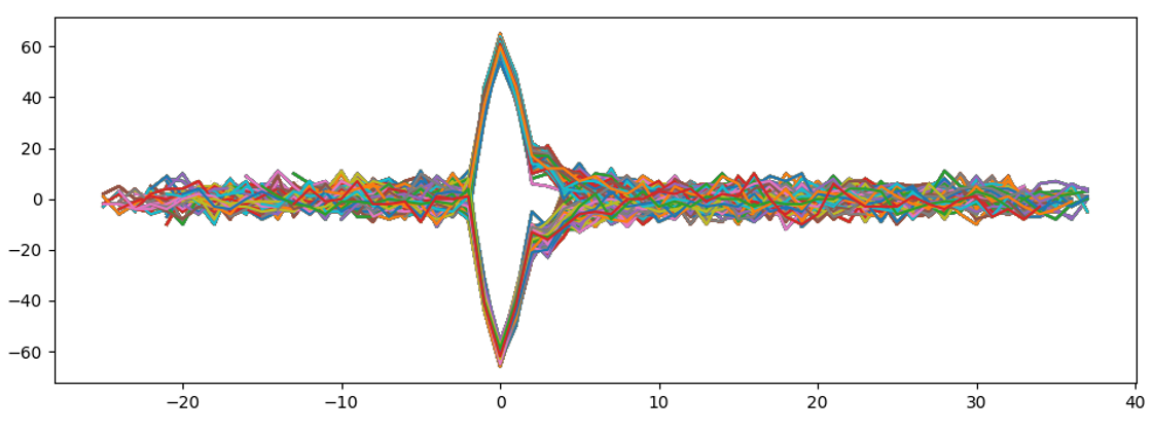



Figure 1.2: Aligned REE for many simulated single bitmisestimates

The situation changes when a bit estimate is wrong. Suppose the bit transmitted was a '1', but it was estimated to be '-1', a situation we will call a '+' error. In that case the input contains channel's response to a '1', to which we subtract the channel's response to a '-1'. The resulting REE's response is the channel response of an isolated '1' pulse.[2] Since we are assuming that all the other bits are correctly predicted, their contributions have been removed, which is why we see an isolated bit response. The REE signal inverts sign when a '-1' signal is misestimated as a '1', which we call a '-' error. Figure 1.2 shows the REE for many simulated single bit misestimates which have been aligned on top of each other. The positive and negative error patterns are clearly visible as is the input referred noise.

[2]While it might seem like this should have twice the amplitude of a 1, the output of the transmitter transitions from '-1' to '1', so it too has an amplitude of 2

While we are assuming that misestimates are not that common (less than 1 %), we can't assume that error clusters will be rare. We use a FFE to compensate for the filtering in the channel before slicing to create the bit estimates. The FIR filter in the FFE correlates the noise in the output samples. If a large noise occurs on one input sample, it can corrupt many of the following bits. Given the generally high-pass nature of the FFE, this means that the likely errors will alternate signs, a '+' error is likely to follow a '-' error and vice versa. Additionally, even when the error rate is abysmal ($10^{-2}$), most of the time there are no errors in the bitstream so there are no errors in the REE. Errors are correlated and bursty. Our initial analysis of our checker will ignore how the FFE effects the probability of errors to simplify the discussion and because there are multiple ways to estimate the bit stream. This discussion will develop a set of error patterns which need to be checked for good performance. However, later we will show that some of these error patterns are extremely unlikely using an FFE-based bit estimator, and thus don't need to be checked in a FFE system.

This approach can be used to check both binary and PAM4 systems, since the qualities of the REE are almost the same for these systems. The results of estimation errors in both systems generate channel pulse responses in the REE. Like in a binary system, with PAM4, the REE signal is the channel response of a pulse with an amplitude equal to the difference between the actual input and the estimated level. The main difference between checking binary and PAM4 systems is the number of possible errors that need to be checked. In a binary system a '-1' estimate can only have a '+' error and a '1' can only have a '-' error. In a PAM4 system (-3,-1, 1, 3), both the '1' and '-1' levels can have either type of error, so an error detector will need to check both cases.

## 1.2 REE Error Detection

Our checker uses the REE as input, and produces a flag at any bit position which it thinks might be in error. Like in most high-speed links, we assume that each system cycle we are given many REE samples to process (16-64). Our error detector

replicates the same hardware[3] at each symbol position to produce the error flag, and all these checkers work in parallel. Since we know that misestimates will be bursty, when an error is detected, that symbol, and many of the following symbols (order 16) will be sent to the Viterbi hardware to generate the correct bits. Thus the critical task for the checker is to accurately detect where the first error occurs.

The rest of this section describes the operation of the hardware checker at each symbol position. Our strategy for this task is quite simple: we check if choosing an alternative set of symbol values, called the check pattern, will lower the energy in the REE. If making these symbol value changes lowers the energy in the REE, then the original estimates were likely in error, and we should raise the check flag and regenerate the bits. Given this basic approach, there are two remaining questions we need to answer: how to create efficient hardware to compute this energy difference, and how to select which check patterns we need to test. The rest of this chapter will address the first question, and Chapter 2 addresses which patterns are needed to match MLSD detectors.

We first look at what REE energy metric to use. Since most lossy channels distribute the energy of a bit over many bit positions, to improve our noise margins for these channels, our hardware will compute the REE energy in the pre-cursor, cursor, and post-cursor bit positions. That is, like many sequence detectors, we sum the energy in these three bit positions to compute the error, and make our decision on this value, rather than just using the energy at the main cursor, which is done in today's link designs. For a channel with pre and post cursor values about half the cursor, this energy metric improves the detector SNR by 3dB.

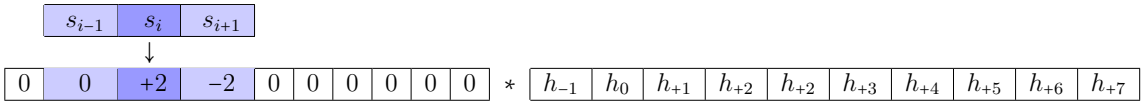



Figure 1.3: The full convolution of a check pattern, $\vec{e}^{\,c}_{(+,-)}$ and the channel response, $\vec{h}$.

To generate the energy differences, conceptually the REE checker convolves each

[3]There is a small difference at the first and last checker because of the edge boundary which can be ignored for this discussion.

check pattern with the channel estimate and subtracts that result from the REE. Figure 1.3 shows the setup of a convolution between a $\vec{e}^{\,c}_{(+,-)}$ check pattern, where the $(+,-)$ indicates that we are checking for '+' error at the cursor, and a '-' error at the post-cursor,[4] and the channel estimate. The symbol currently being checked is highlighted in dark blue, and the sequence of symbols used in that check are highlighted in blue.

| | | $h_{-1}$ | $h_0$ | $h_{+1}$ |
|---|---|---|---|---|
| | | ↓ | | |
| 0 | | $0 * h_0$ | $0 * h_1$ | $0 * h_{+2}$ |
| +2 | → | $+2 * h_{-1}$ | $+2 * h_0$ | $+2 * h_{+1}$ |
| −2 | | | $-2 * h_{-1}$ | $-2 * h_0$ |
| | | ↓ | | |
| | | $2h_{-1}$ | $2(h_0 - h_{-1})$ | $2(h_{+1} - h_0)$ |

Figure 1.4: The partial convolution of $\vec{e}^{\,c}_{(+,-)}$ and $\vec{h}$. The effect of other error patterns can easily be computed by updating the error vector on the left, and then updating the values in the table.

Fortunately, since we are only checking the sequence energy over the pre-, main and post- cursor, we do not need to do this full channel convolution. We only compute the effect of the possible error sequence on these three symbol periods by performing a partial convolution of the channel estimate and our check patterns. Figure 1.4 shows the partial convolution for our $\vec{e}^{\,c}_{(+,-)}$ check pattern.

| | $(i-1)^{th}$ symbol | $i^{th}$ symbol | $(i+1)^{th}$ symbol |
|---|---|---|---|
| REE → | $n_{i-1} + 2h_{-1}$ | $n_i + 2(h_0 - h_{-1})$ | $n_{i+1} + 2(h_{+1} - h_0)$ |
| | | $\downarrow (elements)^2$ | |
| Sym. Energy → | $(n_{i-1} + 2h_{-1})^2$ | $(n_i + 2(h_0 - h_{-1}))^2$ | $(n_{i+1} + 2(h_{+1} - h_0))^2$ |
| | | $\downarrow \sum$ | |
| Seq. Energy → | $(n_{i-1} + 2h_{-1})^2 + (n_i + 2(h_0 - h_{-1}))^2 + (n_{i+1} + 2(h_{+1} - h_0))^2$ | | |

Figure 1.5: Calculating the energy of the REE for a $\vec{e}_{(+,-)}$ error

[4]The first error is always located at the cursor position, with more errors, if they exist, occurring in subsequent positions. Since we check every position, and are looking for the first error, it is ok to assume that the precursor is error free.

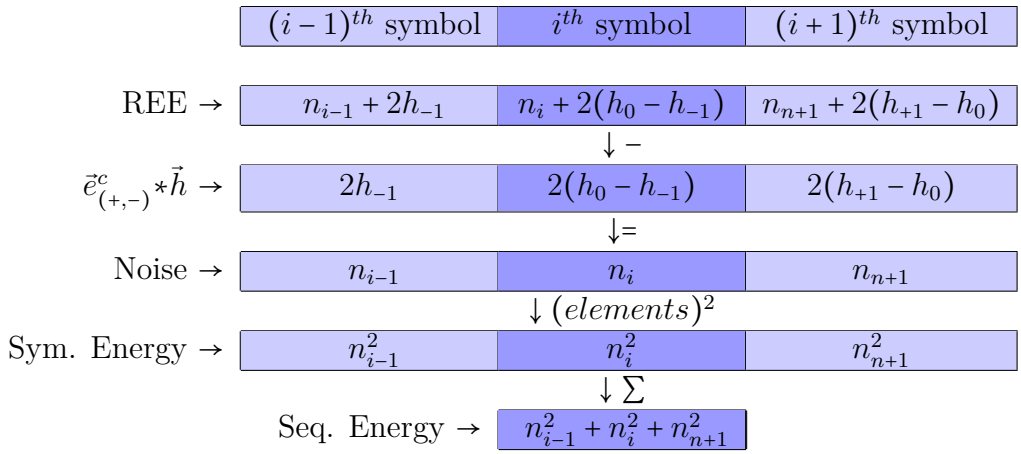



Figure 1.6: Calculating the energy of the REE minus the $\vec{e}^{c}_{(+,-)}*\vec{h}$ for a $\vec{e}_{(+,-)}$ error

These partial results are then subtracted from the REE in the pre-, post-, and cursor positions, and the results are squared and summed to create the check pattern's sequence energy. It is compared against the REE's energy over the same symbols. If the corrected bit pattern is lower energy, the possible error flag is set. For example, let's assume that the estimated bits have a (+,-) error. In that case the REE coming into the checker will consist of the values shown in Figure 1.5, which also gives the total error energy in these three bits. Figure 1.6 shows how the corrected energy is computed: starting with the REE values, the corrections are subtracted, and the results are squared and summed. Eq. 1.2.1 compares the two results:

$$\underbrace{\boxed{n^2_{i-1}+n^2_i+n^2_{i+1}}}_{\text{Checked REE Energy}} \lessgtr \underbrace{\boxed{(2h_{-1}+n_{i-1})^2+(2(h_0-h_{-1})+n_i)^2+(2(h_{+1}-h_0)+n_{i+1})^2}}_{\text{REE Energy}} \quad (1.2.1)$$

Because this is a decision between two options, the result of which is unknown, the sides are equated with a decision operator $\lessgtr$. The left hand side of the comparison is the energy after applying the check pattern. The right hand side is the energy of REE. If the energy is lower on the left hand side of this equation, the checker raises a flag and this region of the input is sent to a Viterbi decoder to be reevaluated. In general there will be a number of check patterns which the unit will need to evaluate. Each pattern will require a separate hardware path to allow all the computation to be done in parallel. If any of the patterns reduce the REE energy, the flag is set.

This entire process is feed-forward and has no data dependencies, which means it is easy to implement as a pipelined digital system.

The entire computation for each check pattern can be nicely summarized in a table, as shown in Figure 1.7. To keep this table generic, the error sequence contains potential error $\alpha$, $\beta$ and $\gamma$ and a checker error sequence a, b and c. These variables can take the values of +1 for a '+' error, 0 for no error, or -1 for a '-' error. Notice that the check pattern reduces the total sequence energy when it's error pattern matches the errors in the REE.

| Generic Residual Estimated Error (REE) Sequence Energy Diagram | | | | |
|---|---|---|---|---|
| Error | $\{\alpha, \beta, \gamma\}$ | Check Pattern | | {a,b,c} |
| Remaining Errors | | $i^{th}$ Precursor | $i^{th}$ Cursor | $i^{th}$ Postcursor |
| $e_i$ | $(\alpha - a)$ | $(\alpha - a)h_{-1}$ | $(\alpha - a)h_0$ | $(\alpha - a)h_{+1}$ |
| $e_{i+1}$ | $(\beta - b)$ | | $(\beta - b)h_{-1}$ | $(\beta - b)h_0$ |
| $e_{i+2}$ | $(\gamma - c)$ | | | $(\gamma - c)h_{-1}$ |
| Noise | | $n_{i-1}$ | $n_i$ | $n_{i+1}$ |
| Sequence Energy | | $((\alpha - a)h_{-1} + n_{i-1})^2 + ((\alpha - a)h_0 + (\beta - b)h_{-1} + n_i)^2 + ((\alpha - a)h_{+1} + (\beta - b)h_0 + (\gamma - c)h_{-1} + n_{i+1})^2$ | | |

Figure 1.7: A Generic REE Sequence Energy Diagram.
Includes the Effects Of Applying The check pattern

# Chapter 2

# Check Patterns

We want to create a set of check patterns where the probability of an error escape is the the same as for an MLSD checker. When this occurs, we claim that these check patterns "cover" the error space. There are two types of coverage we will consider when analyzing the performance of a REE-based checker. Complete coverage for a channel occurs when the REE-based checker achieves equivalent noise margins to a MLSD-based checker for all types errors. While this metric is easy to understand, we will show that it is conservative and will require more check patterns. We can describe the concept of complete coverage in terms of the two types of detection, complete and partial detection, which we introduce next.

We can reduce the number of check patterns required for a particular channel by allowing the checker to have less than MLSD margins for errors that have high initial margins. This is acceptable as long as the checker's worst case error margin is larger than the MLSD checker's worst-case error margin, since the worst-case error margin sets the worst detection probability of the checker. If the worst-case margins are the same, the probability of an error escaping the checkers will be very similar. We say the check patterns have effective coverage for a given channel, when a checker with that set of check patterns achieves equivalent worst-case error margins to a MLSD checker for that channel. We will later further reduce the check patterns needed by taking into account the probability that an error sequence occurs – if probability of that error sequence is very small, it too doesn't need to be corrected.

## 2.1 Complete Detection

When analyzing coverage, its important to realize that the checker performs two types of detection. When our checker contains a check pattern that completely removes the error in the REE, the checker achieves an MLSD equivalent outcome. This detection is a complete detection. If the checker does not contain the error and only partially removes the error from the REE, the checker achieves a sub-optimal decision which is equivalent to the MLSD outcome with an unchecked error penalty. This is called a partial detection and will be discussed in the next section.

In complete detection, we eliminate the error after applying that check pattern. For example, if we are checking an error composed of a positive error followed by a negative error ($\vec{e}_{(+,-)}$) and our checker contains that check pattern ($\vec{e}^{\,c}_{(+,-)}$) then we can write a decision diagram (Figure 2.1) composed of tables like the one shown in Figure 1.7.

| REE Sequence Energy Diagram | | | | |
|---|---|---|---|---|
| Error | {+, -, 0} | Check Pattern | | {+, -, 0} |
| $e_i$ | 0 | | | |
| $e_{i+1}$ | 0 | | | |
| $e_{i+2}$ | 0 | | | |
| Noise | | $n_{i-1}$ | $n_i$ | $n_{i+1}$ |
| Seq. Ener | | $n_{i-1}^2 + n_i^2 + n_{i+1}^2$ | | |

$\lessgtr$

| REE Sequence Energy Diagram | | | | |
|---|---|---|---|---|
| Error | {+, -, 0} | Check Pattern | | {0, 0, 0} |
| $e_i$ | 2 | $2h_{-1}$ | $2h_0$ | $2h_1$ |
| $e_{i+1}$ | -2 | | $-2h_{-1}$ | $-2h_0$ |
| $e_{i+2}$ | 0 | | | |
| Noise | | $n_{i-1}$ | $n_i$ | $n_{i+1}$ |
| Seq. Ener | | $(2h_{-1} + n_{i-1})^2 + (2(h_0 - h_{-1}) + n_i)^2 + (2(h_1 - h_0) + n_{i+1})^2$ | | |

Figure 2.1: Decision diagram for a complete detection of a $\vec{e}_{(+,-)}$ error

The decision equation for the completely checked $\vec{e}_{(+,-)}$ error is derived by comparing the two tabulated total energies at the bottom of these tables:

$$n_{i-1}^2 + n_i^2 + n_{i+1}^2 \lessgtr (2h_{-1} + n_{i-1})^2 + (2(h_0 - h_{-1}) + n_i)^2 + (2(h_{+1} - h_0) + n_{i+1})^2 \quad (2.1.1)$$

Since the noise terms are the same on both sides of the equation, we can expand the terms and simplify, which yields the following inequality for an error to occur:

$$-4n_{i-1}h_{-1} - 4n_i(h_0 - h_{-1}) - 4n_{i+1}(h_{+1} - h_0) > 4h_{-1}^2 + 4(h_0 - h_{-1})^2 + 4(h_{+1} - h_0)^2 \quad (2.1.2)$$

Intuitively, this equation shows that for our checker to fail, the noise terms on

the left hand side need to be, on average, greater than and in opposite direction to the uncorrected error's pulse response. For uncorrelated noise, this is very unlikely to occur as it requires three samples of noise to be in the same direction and those samples of noise have to all be consistently large. Additionally, the right hand side of Eq. 2.1.2 is proportional to the signal energy of the error's pulse response (Eq. 2.1.3).

$$\text{Total Signal Energy} = 4h_{-1}^2 + 4(h_0 - h_{-1})^2 + 4(h_{+1} - h_0)^2 \tag{2.1.3}$$

Beyond intuition, this equations enables the calculation of the probability that the checker fails to detect the error. By assuming that all the noise samples are uncorrelated Gaussian noise with variance $\sigma^2$, the probability that this inequality holds is equal to the Q-function of the error signal power divided by the root-mean-square of the noise coefficients times $\sigma$ (Eq. 2.1.4). The Q-function is a monotonically decreasing function of its argument. Which means when the ratio between the deterministic signal energy and weighted noise increases, in other words the signal-to-noise ratio increases, the probability of an error decreases. At the extremes of the Q-function, this decrease is exponential so small increases in signal energy lead to large decreases in error probability.

$$Pr(\text{Eq. 2.1.2}) = Q\left(\frac{4h_{-1}^2 + 4(h_0 - h_{-1})^2 + 4(h_{+1} - h_0)^2}{\sigma\sqrt{16h_{-1}^2 + 16(h_0 - h_{-1})^2 + 16(h_{+1} - h_0)^2}}\right) \tag{2.1.4}$$

By dividing like terms, Eq. 2.1.4 reduces to a form that shows cleanly that the probability of the detection failure is a direct function of the error's signal to noise ratio, Eq. 2.1.5. More importantly, this probability equivalent to the probability that a three-symbol MLSD detector would detect this error pattern given our bit estimates. So for complete detections, our error checker is equivalent to a MLSD-based detector.

$$Pr(\text{Eq. 2.1.2}) = Q\left(\frac{\sqrt{h_{-1}^2 + (h_0 - h_{-1})^2 + (h_{+1} - h_0)^2}}{\sigma}\right) \tag{2.1.5}$$

In later sections, we will refer to terms like $\frac{\sqrt{h_{-1}^2+(h_0-h_{-1})^2+(h_{+1}-h_0)^2}}{\sigma}$ as the normalized SNR($\sigma$). In this case, this is the normalized SNR($\sigma$) for checking a $\vec{e}_{(+,-)}$ error. The normalized SNR($\sigma$) is very useful as it contains all the important phenomena of

the checker's detection probability. We will show next that it is very useful in computing the performance "loss" when the check pattern doesn't contain the complete error sequence.

## 2.2 Partial Detection

For a partial detection, none of our check patterns completely eliminate the error and the checker now has additional error energy after applying the check pattern.We will also show that this penalty is equal to the cross correlation between the best check pattern and the errors that pattern does not cover.

In the previous section, we walked through an analysis of our checker's performance when checking a $\vec{e}_{(+,-)}$ error. By assuming the actual bit estimate error was +,-,+, adding an additional positive error at the second post cursor position, we can explore the effects of partial detection. Using the same table-based approach (Figure 2.2), we can calculate the two energies the checker will decide between. In this case, we will use the same $\vec{e}_{(+,-,0)}$ check pattern. Pulling out the two decision energies, we can write the decision equation for our partial detection (Eq. 2.2.1).

| REE Sequence Energy Diagram | | | | |
|---|---|---|---|---|
| Error | {+, -, +} | Check Pattern | | {+, -, 0} |
| $e_i$ | 0 | | | |
| $e_{i+1}$ | 0 | | | |
| $e_{i+2}$ | 2 | | | $2h_{-1}$ |
| Noise | | $n_{i-1}$ | $n_i$ | $n_{i+1}$ |
| Seq. Ener | | $n_{i-1}^2 + n_i^2 + (2h_{-1} + n_{i+1})^2$ | | |

$\lessgtr$

| REE Sequence Energy Diagram | | | | |
|---|---|---|---|---|
| Error | {+, -, +} | Check Pattern | | {0, 0, 0} |
| $e_i$ | 2 | $2h_{-1}$ | $2h_0$ | $2h_1$ |
| $e_{i+1}$ | -2 | | $-2h_{-1}$ | $-2h_0$ |
| $e_{i+2}$ | 2 | | | $2h_{-1}$ |
| Noise | | $n_{i-1}$ | $n_i$ | $n_{i+1}$ |
| Seq. Ener | | $(2h_{-1} + n_{i-1})^2 + (2(h_0 - h_{-1}) + n_i)^2 + (2(h_1 - h_0 + h_{-1}) + n_{i+1})^2$ | | |

Figure 2.2: Decision Diagram for a partial detection of a $\vec{e}_{(+,-,+)}$ error

$$n_{i-1}^2 + n_i^2 + (2h_{-1} + n_{i+1})^2 \lessgtr (2h_{-1} + n_{i-1})^2 + (2(h_0 - h_{-1}) + n_i)^2 + (2(h_{+1} - h_0 + h_{-1}) + n_{i+1})^2 \tag{2.2.1}$$

We can manipulate Eq. 2.2.1 to pull out terms common to the complete detection equation (Eq. 2.1.2). The leftover terms represent the penalty for not correcting the positive error at the $(i+2)^{th}$ symbol position. To make the equation more compact, we have gathered common terms between Eq. 2.1.2 and Eq. 2.2.1 into two energy

terms: the total noise energy is represented by $E_n$ (Eq. 2.2.2, the lhs of Eq. 2.1.2) and the total error signal energy $E_s$ (Eq. 2.2.3, the rhs of Eq. 2.1.2).

$$E_n = -4h_{-1}n_{i-1} - 4(h_0 - h_{-1})n_i - 4(h_{+1} - h_0)n_{i+1} \tag{2.2.2}$$

$$E_s = 4h_{-1}^2 + 4(h_0 - h_{-1})^2 + 4(h_{+1} - h_0)^2 \tag{2.2.3}$$

Now we substitute these energy terms into Eq. 2.2.1 and gathering the remaining error energy terms on the right hand side. We find an expression that shows the energy penalty for not correcting the trailing positive error:

$$E_n \lessgtr E_s - 4(h_0 - h_1)h_{-1} \tag{2.2.4}$$

Eq. 2.2.4 shows the penalty for the uncorrected error changes our margin by the cross term of $-4(h_0 - h_1)h_{-1}$. This term is always the sum of the correlation terms between the channel response of the corrected errors (our check pattern) and the uncorrected errors in the three bits where we measure the energy. Since the cursor is larger than postcursor, this term is negative, and decreases our margin. Defining $M_p$ to be the normalized cross correlation between our check pattern and the uncorrected error, $\frac{-4(h_0-h_1)h_{-1}}{\sqrt{E_s}}$, the normalized SNR($\sigma$) equation, Eq. 2.1.5, becomes:

$$Pr(\text{Eq. 2.2.4}) = Q\left(\frac{\sqrt{h_{-1}^2 + (h_0 - h_{-1})^2 + (h_{+1} - h_0)^2} + M_p}{\sigma}\right) \tag{2.2.5}$$

which makes it clear that when the cross correlation is negative, the margin of the check decreases.

Since the cross correlation changes the margin, if the cross correlation is positive the margin will increase. These partial detections have lower error escapes than the base error, and no additional check patterns are required. We can demonstrate this phenomenon by checking a $\vec{e}_{(+,0,+)}$ error using a $\vec{e}^{\,c}_{(+)}$ correction pattern. The only cross-correlation for the residual error (in bit $i+2$) and the corrected error (in bit $i$) occurs in the postcursor position and is $4h_1h_{-1}$. Since this is positive, the margin in this case is larger than checking a isolated single bit error, and doesn't need to be checked directly. The complete tables for this case are given in Appendix C.

## 2.3 Single Precursor Channel Coverage

To generate check patterns, we start by looking for a set of patterns which can provide complete coverage for a single precursor channel. We can simplify this search from covering all possible errors to covering the set of unique errors. The majority of error patterns are not unique. Out of the possible 26 errors[1] , we can eliminate 13 of the possible errors as they equivalent to other errors with a sign flip. Another four errors are eliminated as they represent shifted versions of other errors. This leaves us to deal with a set of nine unique errors to achieve complete coverage. These error cases are shown in Figure 2.4.

The first check pattern we add is (+), which corrects a single bit error.[2] For this pattern, we calculated the cross correlation terms between that error and the unique errors. After this iteration, we found that we would need to add the pattern (+,-) to eliminate several negative cross-correlation terms. So we add that pattern and iterated once more. Now, we found that we needed to add a (+,-,+) pattern to eliminate another set of negative cross-correlation terms. After this, we only had one error that didn't achieve a semi-positive cross correlation term for all semi-positive channels. We can achieve complete coverage by explicitly adding this check pattern, but will show in the next section why this isn't necessary. Figure. 2.3 shows our check patterns.

| Patterns | Cursor | Postcursor | 2nd Postcursor |
|---|---|---|---|
| $\vec{e}_{(+,0,0)}$, $\vec{e}_{(-,0,0)}$ | +/- | | |
| $\vec{e}_{(+,-,0)}$, $\vec{e}_{(-,+,0)}$ | +/- | -/+ | |
| $\vec{e}_{(+,-,+)}$, $\vec{e}_{(-,+,-)}$ | +/- | -/+ | +/- |

Figure 2.3: Set of check patterns For a Single Precursor Channel

[1]There are three bit positions which affect our error signal (i,i+1,i+2) and each of these positions can have no error, or a + or - error. This gives 27 possible cases, one of which is the no error case, which leaves 26 error situations to check.

[2]In this discussion we will use a +, to indicate two checks, one for a + error, and one for a - error. We use this nomenclature to make the correlations between bits explicit. A +,- pattern only generates two patterns, +,- and -,+ .

| Unique Errors | Best check pattern | Cross Correlation | Coverage Constraint |
|---|---|---|---|
| $\vec{e}_{(+,+,+)}$ | $\vec{e}^{c}_{(+,0,0)}$ | $h_1h_{-1} + h_1(h_0 + h_{-1})$ | $h_i > 0$ |
| $\vec{e}_{(+,+,0)}$ | $\vec{e}^{c}_{(+,0,0)}$ | $h_0h_{-1} + h_1h_0$ | $h_i > 0$ |
| $\vec{e}_{(+,+,-)}$ | $\vec{e}^{c}_{(+,0,0)}$ | $h_1h_0$ | $h_i > 0$ |
| $\vec{e}_{(+,0,+)}$ | $\vec{e}^{c}_{(+,0,0)}$ | $h_1h_{-1}$ | $h_i > 0$ |
| $\vec{e}_{(+,0,0)}$ | $\vec{e}^{c}_{(+,0,0)}$ | | |
| $\vec{e}_{(+,0,-)}$ | $\vec{e}^{c}_{(+,0,0)}$ | $-h_1h_{-1}$ | Effective |
| $\vec{e}_{(+,-,+)}$ | $\vec{e}^{c}_{(+,-,+)}$ | | |
| $\vec{e}_{(+,-,0)}$ | $\vec{e}^{c}_{(+,-,0)}$ | | |
| $\vec{e}_{(+,-,-)}$ | $\vec{e}^{c}_{(+,-,0)}$ | $(h_0 - h_1)h_{-1}$ | $h_i > 0$ |

Figure 2.4: Effective Coverage Table for Single Precursor Channel

## 2.4 Effective Coverage

In the previous discussion, we showed that our checker achieves complete coverage for all but one of the error cases. While we could add an additional check pattern to match this error, removing this issue, there is a better solution: for a channel, we can exactly calculate worst-case normalized SNR of detection for a perfect checker. If our worst-case SNR is higher or equal to the worst-case SNR of a perfect checker, then we say our checker achieves effective coverage for that channel. Effective coverage is a more holistic form of coverage.

| Unique Errors | Normalized SNR(1) | | | | | |
|---|---|---|---|---|---|---|
| Channel Tap | $h_{-1}$ | $h_1$ | $h_{-1}$ | $h_1$ | $h_{-1}$ | $h_1$ |
| Channel Value | 0.2 | 0.25 | 0.6 | 0.8 | 0.1 | 0.3 |
| $\vec{e}_{(+,+,+)}$ | 1.10 | | 1.10 | | 1.13 | |
| $\vec{e}_{(+,+,0)}$ | 1.05 | | 1.00 | | 0.94 | |
| $\vec{e}_{(+,+,-)}$ | 1.00 | | 0.90 | | 0.77 | |
| $\vec{e}_{(+,0,+)}$ | 0.63 | | 0.56 | | 0.66 | |
| $\vec{e}_{(+,0,0)}$ | 0.57 | | 0.45 | | 0.50 | |
| $\vec{e}_{(+,0,-)}$ | 0.52 | | 0.44 | | 0.34 | |
| $\vec{e}_{(+,-,+)}$ | 0.54 | | 0.31 | | 0.23 | |
| $\vec{e}_{(+,-,0)}$ | 0.66 | | 0.37 | | 0.21 | |
| $\vec{e}_{(+,-,-)}$ | 0.79 | | 0.49 | | 0.30 | |
| Worst SNR - Perfect Checker | 0.54 | | 0.31 | | 0.21 | |

Figure 2.5: Effective Coverage Table for Single Precursor Channel

To explore effective coverage, Figure 2.5 shows the normalized SNR(1) values for

three very different channels, for each type of error sequence, assuming our checker uses only three check patterns mentioned in the previous section. For each channel (specified by its pre and post cursor tap values; the cursor is always set to 1), we also calculated the worst case SNR for a perfect checker and tabulated it at the bottom of Figure 2.5. We've colored any normalized SNR lower than the SNR of the perfect checker in red. For two of the three channels, the checker achieves effective coverage. For the failing channel, the SNR(1) of the $\vec{e}_{(+,0,-)}$ error, 0.52, is lower than the worst-case SNR of a perfect checker, 0.54. While this difference means that our checker doesn't achieve effective coverage for this channel, there are a few reason why this might be acceptable. First, the decrease is margin is very small. Second, this situation only occurs when the pre and post cursor are very small, which means that the channel margin is already very high. For these channels a small decrease in margin won't matter. Finally as we will show in the next chapter, a +,0,- error is unlikely to be generated from a FFE based bit estimator.

This answer is not entirely satisfactory as it reveals how dependent the checker's coverage is on the specific channel. We can get a broader view of our checker's effective coverage by taking the core of this analysis - comparing the lowest normalized SNR(1) of our checker to the lowest normalized SNR(1) of a perfect checker - and performing it over a wide range of pre and post cursor values. Instead of showing these results in a large table, we can create two plots, shown in Figure 2.6, one of the worst case SNR(1) of the perfect checker and one of the ratio of our checker's worst case SNR(1) and the perfect checker's worst case SNR(1). In these plots the values are displayed in decibels so the values of ratio'd plot can be directly added to the plot of the perfect checker, recovering the worst SNR(1) of the checker with the limited set of patterns. For the left plot, white represents a pre and post cursor combination that we did not check. For the right plot, the white represents no deviation from perfect checker performance. We did not check every value of pre and post cursor as some combinations produce either channels that have large loss (>40 dB).[3]We also did not compute the effective coverage for channels that had precursors larger than the post cursors.

[3]We calculate loss by extending the tail of the channel with a decaying exponential of 0.6

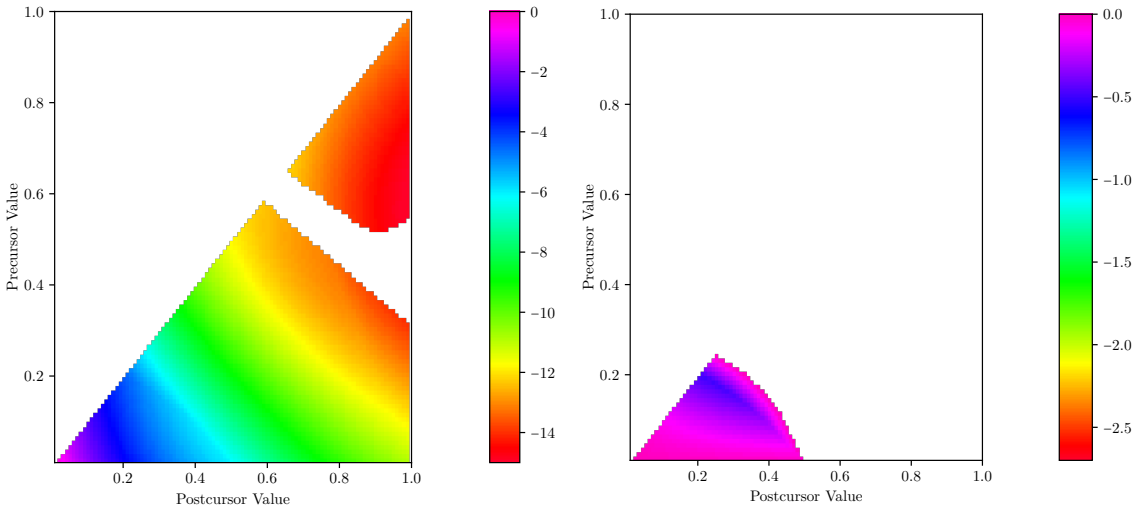



Figure 2.6: Normalized SNR(1) Of Worst Case Errors.
$h_{-1}$ and $h_1$ normalized by $h_0$

As our previous table demonstrated, the ratio'd plot shows that our checker deviates from the perfect checker for low values of pre and post cursors. In this region, the checker pays a minor performance penalty of approximately 0.5 dB for not checking the $\vec{e}_{(+,0,-)}$ error. This matches both our previous calculation and our intuition, since we determined that our checker did not completely cover this error. We expect to pay a penalty for this error in certain regions.

Fortunately, this region has very high SNR(1) overall. The penalty of 0.5 dB - a reduction in margin by 5% is acceptable. It is not worth the additional hardware to improve the checker's margins. One can calculate our checker's worst case SNR(1) by adding the debical values for both plots together.

We will call the region in the lower left of the relative SNR plot - a hot spot. In the two precursor section, we will show that there can be multiple hot spots, each related to an unique error that is not completely covered. Like we did for the hot spot in this analysis, we can determine if a particular hot spot is worth the additional hardware cost to cover the pattern that causes that hot spot. If the hot spot falls within a region where the perfect checker has high SNR(1) then we know that margin loss will not dramatically impact the error rate. If a hot spot falls within a region

where the perfect checker has low SNR(1) then we know we should consider adding that pattern to our check pattern set. This hot spot analysis serves as a vehicle to iteratively choose check patterns to achieve effective coverage, similar to how we used cross correlation terms as a vehicle to iteratively choose check patterns to achieve near complete coverage.

We start with some initial guess of check patterns and calculate the relative performance penalty. We look for patterns that cause hot spots in low SNR(1) regions and add those check patterns until we are satisfied with our checker's SNR(1). Since the needed check patterns depend on the channel, our detector has programmable check patterns. Thus the hardware only need to have hardware to support the number of check patterns for the worst specific channel, and not the number of check patterns that might be needed by any channel.

## 2.5 Two Precursor Channel Coverage

| Patterns | Cursor | Postcursor | 2nd Postcursor | 3nd Postcursor |
|---|---|---|---|---|
| $\vec{e}_{(+,0,0,0)}$, $\vec{e}_{(-,0,0,0)}$ | +/- | | | |
| $\vec{e}_{(+,-,0,0)}$, $\vec{e}_{(-,+,0,0)}$ | +/- | -/+ | | |
| $\vec{e}_{(+,-,+,0)}$, $\vec{e}_{(-,+,-,0)}$ | +/- | -/+ | +/- | |

Figure 2.7: Set of check patterns For a Two Precursor Channel

We initially performed the same cross-correlation based coverage analysis that we did for a single precursor channel for a two precursor channel. We quickly found that with a limited set of check patterns, complete coverage for a two precursor channel is out of reach. For a two precursor channel, the number of unique errors triples from nine to 27. For nine out of these 27 unique errors, the checker does not achieve clear complete coverage. For these nine cases, there are multiple cross correlation terms, some of which are contradictory.

In essence, we weren't going to find a straightforward answer and it was difficult to determine where regions of effective coverage would exist. So we expanded our previous graphical extended effective coverage analysis to two precursor channels.

We started with our set of check patterns set to the ones we derived previously for single precursor channels. But we extended the calculation space to include the second precursor, $h_{-2}$. Initially, to make the analysis tractable and plottable, we performed this analysis with $h_{-2}$ to set to $0.3h_{-1}$.

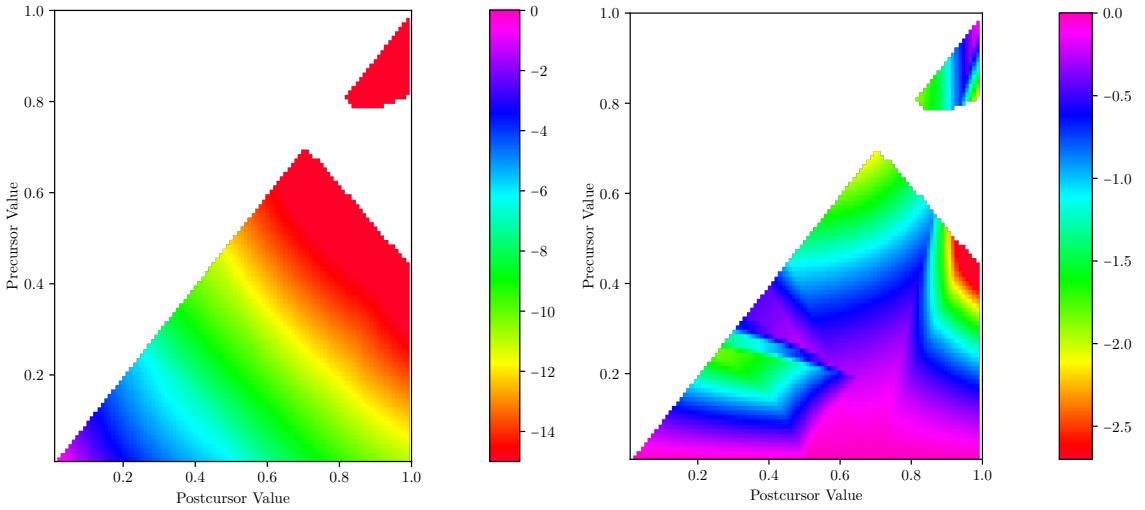



Figure 2.8: Normalized SNR(1) Of Perfect Checker And dB Penalty Of Our check patterns

Figure 2.8 contains the map of the perfect checker performance and the map of relative performance penalty of our checker for two precursor channels. The map of the relative penalty now shows that there are multiple performance penalty hot spots. The hot spot in the bottom left sits in a region of relatively high SNR(1) and is associated with the $\vec{e}_{(+,0,-,-)}$ error. The two hot spots near the top of the triangle are in regions of very low SNR(1) are associated with the errors, $\vec{e}_{(+,-,0,+)}$ and $\vec{e}_{(+,-,-,+)}$.

If we plot and color the worst case error pattern over all the channel values, we can determine which error is limiting the performance of our checker. This plot, Figure 2.9, shows that there are six regions where an additional checker patttern would improve our checker's worst case SNR(1). We exploit our checker's programmable check patterns, to dynamically add that worst case error to our set of check patterns. In this case, our fourth check pattern is a "roaming" pattern, that depends on the channel and the worst case error for that channel.

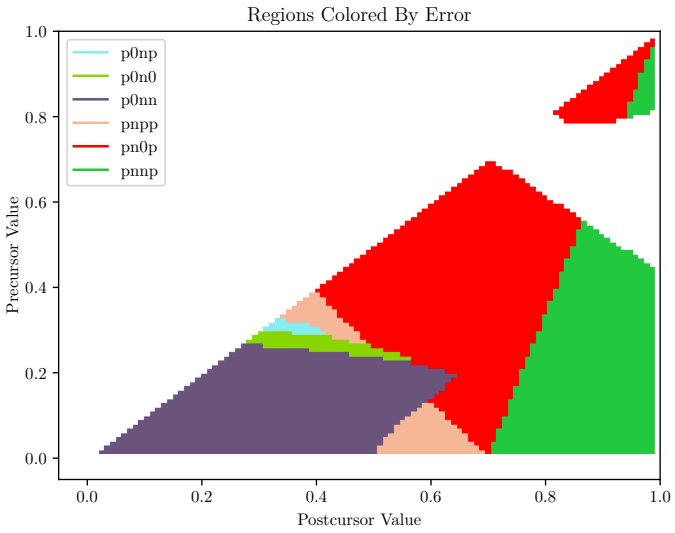



Figure 2.9: Regions Colored By Dominant Worst Case Error

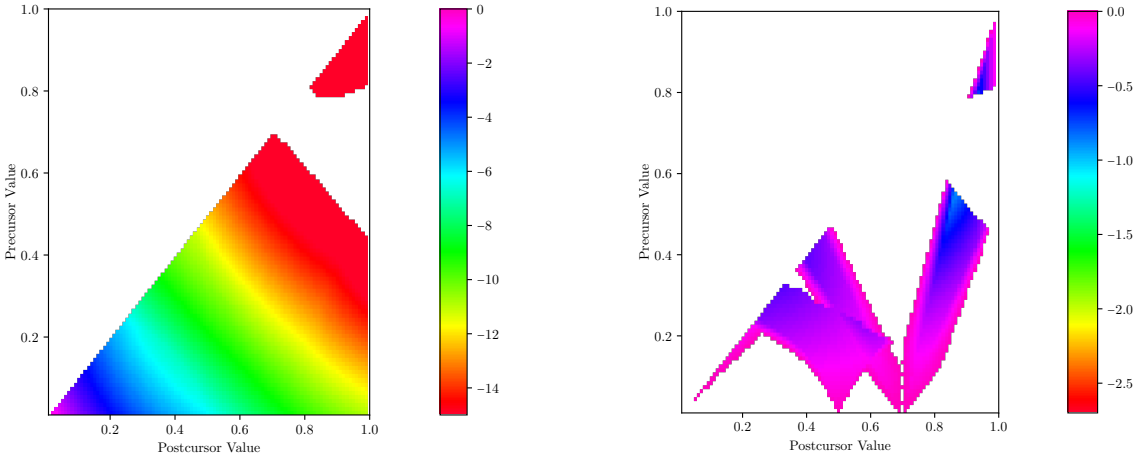



Figure 2.10: Adding $\vec{e}^{c}_{(+,-,0,+)}$ to check patterns to reduce penalty in difficult region

Recalculating the extended effective coverage map accounting for this roaming pattern, produces a new performance penalty map with dramatically lower penalties (Figure 2.10). There are still some regions where the checker achieves a worse SNR(1) than the perfect checker, but this degradation is now generally lower than 0.5 dB. If we add an additional roaming pattern to cover these errors, we would further reduce the degradation but with diminishing returns.

The normalized SNR plots give the detector margins **should** this error sequence occur, but says nothing about the probability of this error sequence being generated. In our actual system, we use a FFE and slicer to generate our bit estimate. The FFE correlates noise which alters the probability that an error pattern occurs. When this correlation is taken into account, the some of the errors that may be difficult to detect become so infrequent that they can be ignored out right. The next chapter explores how our bit estimator implementation biases the probability of errors and changes our check pattern selection.

# Chapter 3

# Check Patterns for Practical Bit Estimators

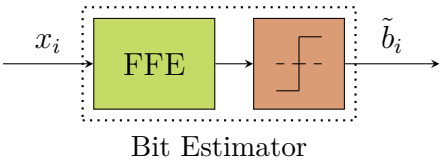



Figure 3.1: Block Diagram Of Bit Estimator

The types of errors that likely to occur are dependent on the implementation of the bit estimator. For our system, we use a feed forward equalizer and slicer to create an estimate of the transmitted bits (Figure 3.1). The main trade off of FFEs is that they reduce ISI at the expense of amplifying and correlating noise. The noise at the input of the slicer is now correlated which means that the errors at the output of the slicer will be correlated. Figure 3.2 shows a simple way to view this. If an impulse of

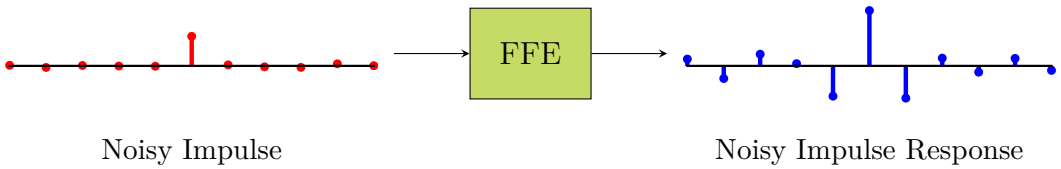



Figure 3.2: Noise being correlated by an FFE

noise enters the FFE, the output of the FFE will be it's impulse response centered on the noisy impulse. If the impulse in the noise is large enough, the output noise over multiple cycles will be very high. For lossy channels, the number of FFE taps increase to compensate for the loss, which drives up the noise at the slicer substantially over multiple symbol periods.

In our previous analysis, we ignored how the errors were generated when determining essential check patterns as there are many methods of producing bit estimates. However, when we use an FFE to generate bit estimates, certain error patterns are substantially less likely to occur. So instead of trying to achieve the best possible margin for the hardest error to detect, for an FFE-based bit estimator, we should try to achieve the highest margin for errors that are likely to occur. To do an analysis that accounts for the FFE's correlation, we need to choose the $\sigma$ of the input noise to the FFE in order to calculate the FFE's filter.[1]

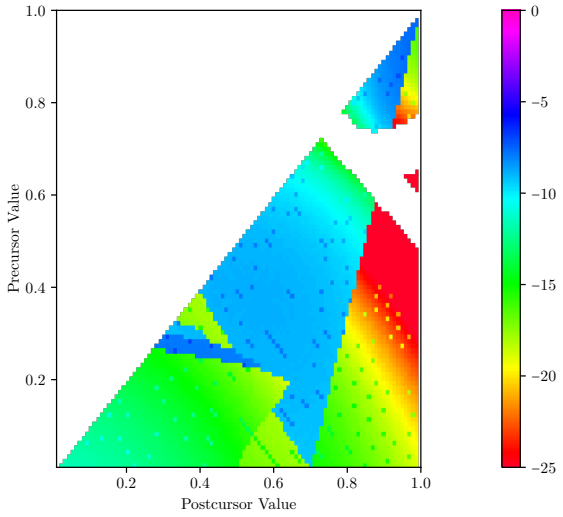



Figure 3.3: Probability Of Worst Case Error Pattern

When we take the errors that had the worst margins in our previous effective coverage analysis and look at the likelihood that they occur for an FFE based bit estimator, we find that many of the errors we were concerned about have probabilities

[1]We use an MSE algorithm to find the FFE coefficients. Since it balances residual ISI with noise, the input noise affect the filter tap values.

substantially below $10^{-15}$ (Figure 3.3). For this analysis, we set the input noise, $\sigma$, at the level which causes a single bit error rate of $10^{-4}$ at the output of the FFE for this channel. This $\sigma$ is different for each channel. Then for each error, we calculate the probability of the worst case error pattern occurring by numerically integrating over quadrants of a multi-dimensional Gaussian with a correlation matrix set by the FFE's filter coefficients. In many cases, the error patterns that mattered for achieving effective coverage, are extremely unlikely to occur. For example, the (+,-,-,+) error that dominated the right area of our SNR(1) plots has a probability of occurring of $10^{-25}$. If we simply failed to detect this error, it would not affect the overall BER of the link.

| | Channel 1 | | | | Channel 2 | | | | Channel 3 | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Channel Properties | | $h_{-1}$ | $h_1$ | $\sigma$ | | $h_{-1}$ | $h_1$ | $\sigma$ | | $h_{-1}$ | $h_1$ | $\sigma$ |
| | | 0.1 | 0.3 | 0.164 | | 0.6 | 0.65 | 0.036 | | 0.5 | 0.95 | 0.022 |
| Unq. Err. | SNR(1) | $R_D$ | $R_E$ | BER | SNR(1) | $R_D$ | $R_E$ | BER | SNR(1) | $R_D$ | $R_E$ | BER |
| $\vec{e}_{(+,+,+,+)}$ | 1.04 | 1.33E-10 | 1.32E-19 | 1.75E-29 | 1.31 | 0.00E+00 | 1.20E-126 | 0.00E+00 | 1.20 | 0.00E+00 | 1.03E-223 | 0.00E+00 |
| $\vec{e}_{(+,+,+,0)}$ | 1.03 | 1.91E-10 | 1.08E-12 | 2.06E-22 | 1.28 | 0.00E+00 | 5.87E-55 | 0.00E+00 | 1.17 | 0.00E+00 | 2.30E-81 | 0.00E+00 |
| $\vec{e}_{(+,+,+,-)}$ | 1.02 | 2.72E-10 | 4.55E-13 | 1.24E-22 | 1.24 | 0.00E+00 | 1.91E-53 | 0.00E+00 | 1.13 | 0.00E+00 | 7.73E-80 | 0.00E+00 |
| $\vec{e}_{(+,+,0,+)}$ | 0.98 | 1.37E-09 | 6.36E-09 | 8.73E-18 | 1.11 | 0.00E+00 | 3.07E-23 | 0.00E+00 | 1.02 | 0.00E+00 | 2.99E-34 | 0.00E+00 |
| $\vec{e}_{(+,+,0,0)}$ | 0.97 | 1.92E-09 | 6.59E-06 | 1.27E-14 | 1.07 | 0.00E+00 | 2.93E-14 | 0.00E+00 | 0.98 | 0.00E+00 | 3.58E-17 | 0.00E+00 |
| $\vec{e}_{(+,+,0,-)}$ | 0.96 | 2.69E-09 | 1.57E-07 | 4.22E-16 | 1.03 | 0.00E+00 | 2.27E-21 | 0.00E+00 | 0.94 | 0.00E+00 | 1.12E-28 | 0.00E+00 |
| $\vec{e}_{(+,+,-,+)}$ | 0.91 | 1.24E-08 | 5.97E-10 | 7.38E-18 | 0.90 | 0.00E+00 | 2.33E-14 | 0.00E+00 | 0.83 | 0.00E+00 | 1.19E-16 | 0.00E+00 |
| $\vec{e}_{(+,+,-,0)}$ | 0.91 | 1.70E-08 | 2.49E-07 | 4.22E-15 | 0.87 | 0.00E+00 | 2.93E-14 | 0.00E+00 | 0.80 | 0.00E+00 | 3.58E-17 | 0.00E+00 |
| $\vec{e}_{(+,+,-,-)}$ | 0.90 | 2.33E-08 | 1.15E-10 | 2.68E-18 | 0.83 | 0.00E+00 | 5.17E-24 | 0.00E+00 | 0.76 | 0.00E+00 | 3.42E-31 | 0.00E+00 |
| $\vec{e}_{(+,0,+,+)}$ | 0.63 | 6.51E-05 | 6.36E-09 | 4.14E-13 | 0.79 | 0.00E+00 | 3.07E-23 | 0.00E+00 | 0.73 | 0.00E+00 | 2.99E-34 | 0.00E+00 |
| $\vec{e}_{(+,0,+,0)}$ | 0.62 | 8.15E-05 | 1.62E-04 | 1.32E-08 | 0.75 | 0.00E+00 | 1.30E-04 | 0.00E+00 | 0.69 | 0.00E+00 | 8.07E-05 | 0.00E+00 |
| $\vec{e}_{(+,0,+,-)}$ | 0.61 | 1.02E-04 | 1.68E-05 | 1.71E-09 | 0.71 | 0.00E+00 | 1.69E-04 | 0.00E+00 | 0.65 | 0.00E+00 | 1.78E-04 | 0.00E+00 |
| $\vec{e}_{(+,0,0,+)}$ | 0.57 | 2.75E-04 | 1.25E-02 | 3.42E-06 | 0.58 | 0.00E+00 | 3.53E-02 | 0.00E+00 | 0.54 | 0.00E+00 | 3.08E-02 | 0.00E+00 |
| $\vec{e}_{(+,0,0,0)}$ | 0.56 | 3.37E-04 | 1.00E+00 | 3.37E-04 | 0.55 | 0.00E+00 | 1.00E+00 | 0.00E+00 | 0.51 | 0.00E+00 | 1.00E+00 | 0.00E+00 |
| $\vec{e}_{(+,0,0,-)}$ | 0.55 | 4.13E-04 | 1.77E-02 | 7.33E-06 | 0.51 | 0.00E+00 | 2.98E-03 | 0.00E+00 | 0.47 | 0.00E+00 | 2.09E-03 | 0.00E+00 |
| $\vec{e}_{(+,0,-,+)}$ | 0.51 | 1.02E-03 | 5.96E-05 | 6.08E-08 | 0.38 | 0.00E+00 | 1.83E-04 | 0.00E+00 | 0.36 | 0.00E+00 | 7.83E-05 | 0.00E+00 |
| $\vec{e}_{(+,0,-,0)}$ | 0.50 | 1.23E-03 | 1.24E-03 | 1.52E-06 | 0.34 | 0.00E+00 | 5.33E-05 | 0.00E+00 | 0.32 | 0.00E+00 | 1.33E-05 | 0.00E+00 |
| $\vec{e}_{(+,0,-,-)}$ | 0.49 | 1.47E-03 | 1.57E-07 | 2.32E-10 | 0.34 | 0.00E+00 | 2.27E-21 | 0.00E+00 | 0.32 | 0.00E+00 | 1.12E-28 | 0.00E+00 |
| $\vec{e}_{(+,-,+,+)}$ | 0.57 | 2.42E-04 | 5.97E-10 | 1.45E-13 | 0.26 | 2.15E-13 | 2.33E-14 | 5.02E-27 | 0.25 | 0.00E+00 | 1.19E-16 | 0.00E+00 |
| $\vec{e}_{(+,-,+,0)}$ | 0.59 | 1.62E-04 | 1.44E-05 | 2.33E-09 | 0.22 | 2.76E-10 | 2.60E-03 | 7.18E-13 | 0.21 | 0.00E+00 | 5.39E-03 | 0.00E+00 |
| $\vec{e}_{(+,-,+,-)}$ | 0.61 | 1.08E-04 | 1.59E-07 | 1.71E-11 | 0.19 | 1.22E-07 | 1.19E-04 | 1.45E-11 | 0.20 | 0.00E+00 | 2.61E-04 | 0.00E+00 |
| $\vec{e}_{(+,-,0,+)}$ | 0.60 | 1.18E-04 | 5.96E-05 | 7.04E-09 | 0.14 | 7.92E-05 | 1.83E-04 | 1.45E-08 | 0.15 | 1.26E-11 | 7.83E-05 | 9.84E-16 |
| $\vec{e}_{(+,-,0,0)}$ | 0.62 | 7.77E-05 | 7.27E-03 | 5.65E-07 | 0.20 | 2.65E-08 | 1.15E-01 | 3.05E-09 | 0.15 | 1.82E-12 | 1.56E-01 | 2.83E-13 |
| $\vec{e}_{(+,-,0,-)}$ | 0.64 | 5.06E-05 | 1.69E-05 | 8.57E-10 | 0.26 | 6.07E-13 | 1.97E-04 | 1.19E-16 | 0.16 | 2.44E-13 | 2.23E-04 | 5.45E-17 |
| $\vec{e}_{(+,-,-,+)}$ | 0.63 | 5.56E-05 | 1.91E-08 | 1.06E-12 | 0.25 | 1.33E-12 | 1.30E-13 | 1.74E-25 | 0.11 | 7.21E-07 | 1.91E-16 | 1.38E-22 |
| $\vec{e}_{(+,-,-,0)}$ | 0.65 | 3.60E-05 | 2.49E-07 | 8.95E-12 | 0.31 | 0.00E+00 | 2.93E-14 | 0.00E+00 | 0.11 | 1.72E-07 | 3.58E-17 | 6.18E-24 |
| $\vec{e}_{(+,-,-,-)}$ | 0.67 | 2.30E-05 | 4.55E-13 | 1.05E-17 | 0.37 | 0.00E+00 | 1.91E-53 | 0.00E+00 | 0.12 | 3.83E-08 | 7.73E-80 | 2.96E-87 |

Figure 3.4: Optimal BER Coverage Table

Instead, we can do an alternative analysis that is FFE-aware. For three channels, we calculated the SNR(1) and the error probability for all the unique errors given our original limited check pattern set (Figure 3.4). Each of these channels represents a channel from a hot spot region on Figure 2.8. To compare between different channels, we set the $\sigma$ for each channel to force a single bit error rate to $5 * 10^{-4}$. For Channel 1, the noise is very large as the channel is low loss. For Channel 2 and 3, the noise

is much smaller as the FFE dramatically amplifies the noise to correct the channel ISI. For each SNR(1), we estimate the overall link's BER by scaling the detection probability ($R_D$) by the relative probability that error pattern will occur ($R_E$). We use a relative probability that an error pattern will occur because in the limit of a perfect channel, the error rate of a single bit error and the detection escape of a single bit error converge on the same probability as there is no advantage to sequence detection. Of these three channels, only one channel 2's BER is set by a pattern not in our initial check pattern set, $\vec{e}_{(+,-,0,+)}$. For this channel, if I add this pattern to my check patterns, the BER of that pattern decreases from $10^{-8}$ to $>10^{-9}$.

It is important to note that for the low loss channel, we've set the noise artificially high to achieve a single bit error rate of $5*10^{-4}$. In a practical link, the noise will likely be substantially lower. For high loss channels, 2 and 3, if one found the error rates not satisfactory ($10^{-9}$ and $10^{-13}$, one could add additional symbols to the checker. This risks a likely increase in the number of check patterns as the number of unique errors required to check expands to 81. Alternatively, one could look at improving the quality of the bit estimate with a limited DFE or DFFE. Whether this would be useful would require a detailed power budget analysis, out of the scope of this chapter.

# Appendices

# Appendix A

# Glossary Of Terms

**Normalized SNR($\sigma$)**

$$\frac{\text{Total Detection Energy Margin}}{\sigma} \tag{A.0.1}$$

**Channel Loss** A channel is reasonable when its worst case loss is greater than 40 dB. We estimate the loss for the total channel, by assuming the tail decays exponentially from the postcursor by a factor 0.6.

$$h_{i+1} = 0.6h_i \text{ for } i > 1 \tag{A.0.2}$$

# Appendix B

# DFE

The table formulation for our checker can also be used to analyze other types of decision based equalization. This appendix applies it to analyze DFE and will allow us to easily compare the error rates between the two approaches.

Decision feedback equalization (DFE), uses past bit decisions to remove ISI at the current symbol decision. Since it uses bit decisions, and not estimates like our checker, DFE is unable to account for future bits, and thus can't cancel any precursor effects. In the framework of our decision tables, the DFE decides between two outcomes - the correct and incorrect symbol decision. If the DFE makes the wrong decision, an error is left in the cursor location. Like our checker, we will assume that the DFE has an accurate channel model with enough taps to cancel out all the postcursor ISI. Additionally, we will assume that there are no incorrect decisions in the past, to compute the probability of creating the "first" errors (since DFE errors can be bursty). This means there are no errors before the current symbol. However at the current symbol decision, the next symbol is unknown and so to compute the error margin we will assume that symbol is always an error.

Fig. B.1 is a decision diagram for the DFE that encompasses all these factors. The left diagram represents the DFE making the correct choice and the right diagram represents the DFE making the wrong choice. Again tables are equated with a decision operator $\lessgtr$ since we don't know the result of the comparison. Unlike the diagrams used for sequence detection, here we only consider the energy at the cursor position.

| Perfect DFE Symbol Energy Diagram - Correct Path | | |
|---|---|---|
| Remaining Errors | | $i^{th}$ Cursor |
| $e_i$ | 0 | |
| $e_{i+1}$ | 0 | |
| $e_{i+2}$ | -2 | $-2h_{-1}$ |
| Noise | | $n_i$ |
| DFE Symbol Energy | | $(-2h_{-1}+n_i)^2$ |

$\lessgtr$

| Perfect DFE Symbol Energy Diagram - Wrong Path | | |
|---|---|---|
| Remaining Errors | | $i^{th}$ Cursor |
| $e_i$ | 0 | |
| $e_{i+1}$ | +2 | $2h_0$ |
| $e_{i+2}$ | -2 | $-2h_{-1}$ |
| Noise | | $n_i$ |
| DFE Symbol Energy | | $(2(h_0-h_{-1})+n_i)^2$ |

Figure B.1: Decision diagram for a perfect DFE

$$(-2h_{-1}+n_i)^2 \lessgtr (2(h_0-h_{-1})+n_i)^2 \tag{B.0.1}$$

We can now pull out the decision equation by comparing between the total DFE symbol energy of both tables (Eq. B.0.1). If the energy on the left hand side of this decision is lower than the energy of the right hand side of this equation, the DFE will make the correct decision. By expanding the square of both sides of the decision equation (Eq. B.0.2) and cancelling like terms (Eq. B.0.3), we derive the intuitive answer. The margin of the DFE decision is limited by the presence of the next decision's precursor.

$$4h_{-1}^2 - 4h_{-1}n_i + n_i^2 < 4(h_0^2 - h_0h_{-1} + h_{-1}^2) + 4(h_0-h_{-1})n_i + n_i^2 \tag{B.0.2}$$

$$n_i < h_0 - h_{-1} \tag{B.0.3}$$

# Appendix C

# +,0+ Error Pattern

| REE Sequence Energy Diagram | | | | |
|---|---|---|---|---|
| Error | {+, 0, 0} | Check Pattern | | {+, 0, 0} |
| $e_i$ | 0 | | | |
| $e_{i+1}$ | 0 | | | |
| $e_{i+2}$ | 0 | | | |
| Noise | | $n_{i-1}$ | $n_i$ | $n_{i+1}$ |
| Seq. Ener | | $n_{i-1}^2 + n_i^2 + n_{i+1}^2$ | | |

$\lessgtr$

| REE Sequence Energy Diagram | | | | |
|---|---|---|---|---|
| Error | {+, 0, 0} | Check Pattern | | {0, 0, 0} |
| $e_i$ | 2 | $2h_{-1}$ | $2h_0$ | $2h_1$ |
| $e_{i+1}$ | 0 | | | |
| $e_{i+2}$ | 0 | | | |
| Noise | | $n_{i-1}$ | $n_i$ | $n_{i+1}$ |
| Seq. Ener | | $(2h_{-1} + n_{i-1})^2 + (2h_0 + n_i)^2 + (2h_1 + n_{i+1})^2$ | | |

Figure C.1

| REE Sequence Energy Diagram | | | | |
|---|---|---|---|---|
| Error | {+, 0, +} | Check Pattern | | {+, 0, 0} |
| $e_i$ | 0 | | | |
| $e_{i+1}$ | 0 | | | |
| $e_{i+2}$ | 2 | | | $2h_{-1}$ |
| Noise | | $n_{i-1}$ | $n_i$ | $n_{i+1}$ |
| Seq. Ener | | $n_{i-1}^2 + n_i^2 + (2h_{-1} + n_{i+1})^2$ | | |

$\lessgtr$

| REE Sequence Energy Diagram | | | | |
|---|---|---|---|---|
| Error | {+, 0, +} | Check Pattern | | {0, 0, 0} |
| $e_i$ | 2 | $2h_{-1}$ | $2h_0$ | $2h_1$ |
| $e_{i+1}$ | 0 | | | |
| $e_{i+2}$ | 2 | | | $2h_{-1}$ |
| Noise | | $n_{i-1}$ | $n_i$ | $n_{i+1}$ |
| Seq. Ener | | $(2h_{-1} + n_{i-1})^2 + (2h_0 + n_i)^2 + (2(h_1 + h_{-1}) + n_{i+1})^2$ | | |

Figure C.2

Like we did in our previous analysis, we start with the two decision diagrams. Figure C.1 is the situation without a residual error, and Figure C.2 contains the addition error. The resulting two decision equations are Eq. C.0.1 and Eq. C.0.2.

$$n_{i-1}^2 + n_i^2 + n_{i+1}^2 \lessgtr (2h_{-1} + n_{i-1})^2 + (2h_0 + n_i)^2 + (2h_1 + n_{i+1})^2 \quad \text{(C.0.1)}$$

$$n_{i-1}^2 + n_i^2 + (2h_{-1} + n_{i+1})^2 \lessgtr (2h_{-1} + n_{i-1})^2 + (2h_0 + n_i)^2 + (2(h_1 + h_{-1}) + n_{i+1})^2 \quad \text{(C.0.2)}$$

We subtract the first equation from the second to find the total difference in energy between our complete and partial detection (Eq. C.0.3).

$$(2h_{-1} + n_{i+1})^2 - n_{i+1}^2 \lessgtr (2(h_1 + h_{-1}) + n_{i+1})^2 - (2h_1 + n_{i+1})^2 \tag{C.0.3}$$

Then we expand and remove common terms, revealing the expected result - the correlation between the our check pattern and the uncorrected errors (Eq. C.0.4).

$$0 \lessgtr 4h_1h_{-1} \tag{C.0.4}$$

# Appendix D

# Smaller $h_{-2}$

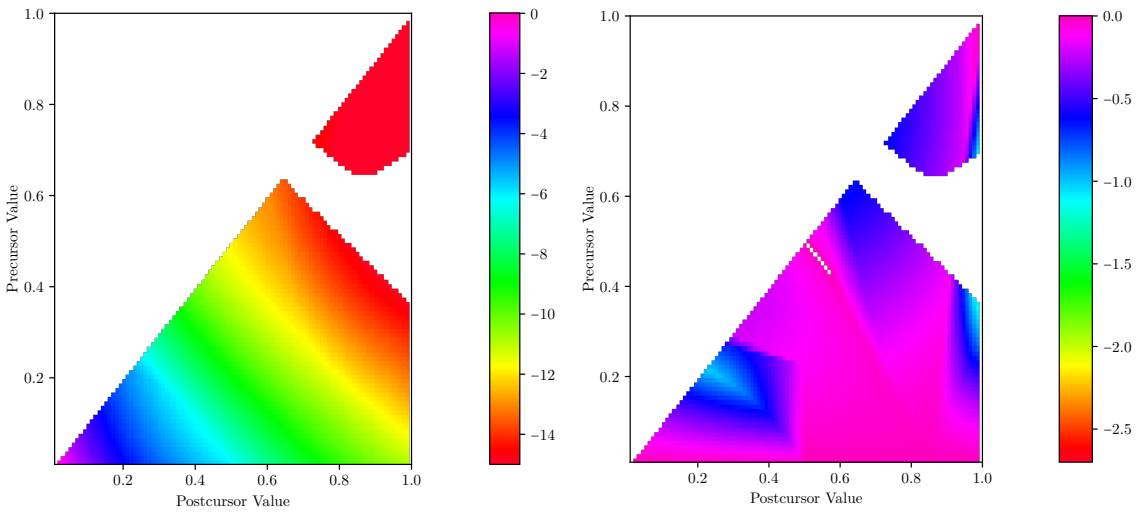



Figure D.1: Hotspot Analysis for $\frac{h_{-2}}{h_{-1}} = 0.15$

We redid the non-roaming analysis with a $h_{-2}$ equal to $0.15h_{-1}$ to see if it changes the hot spots. In this case, (Figure D.1) the penalty of not adding any check patterns is limited to 0.7 dB in the lowest margin regions. This makes sense as we've reduced the size of the second precursor, which moves the channels to look more like single precursor channels.

When we add in the roaming pattern, we see a dramatic increase in channels that achieve effective coverage and the worst deviation is less than 0.5 dB. Similar to the

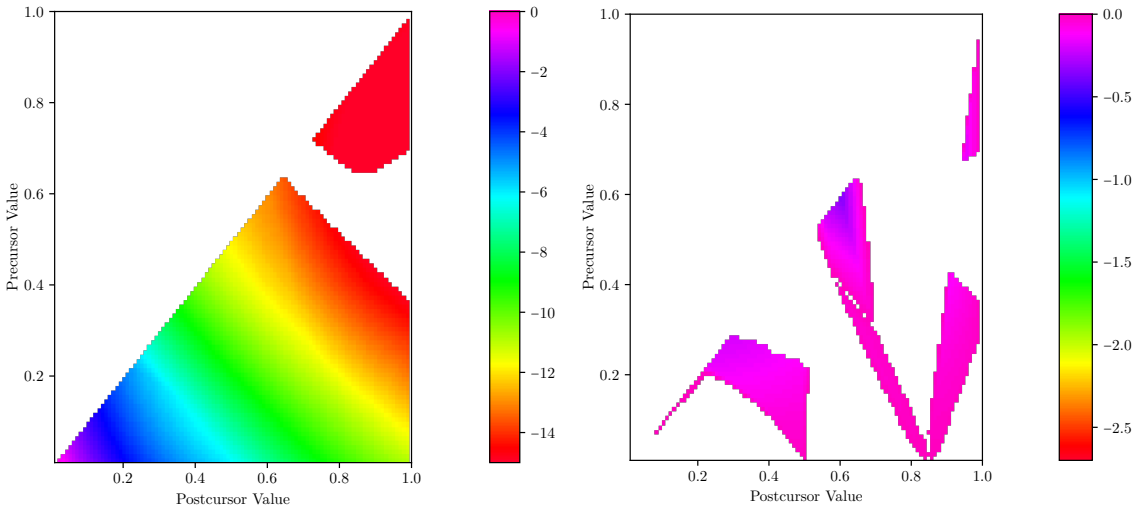



Figure D.2: Adding $\vec{e}^{c}_{(+,-,0,+)}$ to check patterns to reduce penalty in difficult region

previous case, the addition of the roaming pattern uncovers new errors that fail to achieve effective coverage.

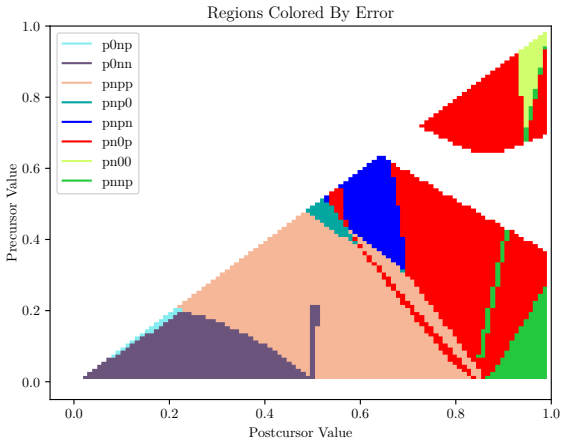



Figure D.3: Regions Colored By Dominant Worst Case Error

# Appendix E

# Modified System

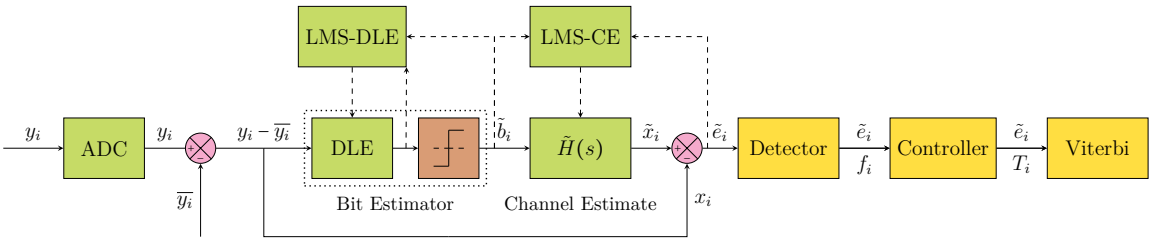



Figure E.1: Block Diagram Of Error Detection Scheme